



3D Dynamic Reconstruction from Multi-view Capture

Semester Thesis

Pelin Kincir

Department of Information Technology and Electrical Engineering

Supervisor: Prof. Ender Konukoglu

Computer Vision Laboratory

Department of Information Technology and Electrical Engineering

Advisors: Prof. Otmar Hilliges
Dr. Chengwei Zheng
Lixin Xue

AIT lab/Institute for Intelligent Interactive Systems (IIS)

Department of Computer Science

June 23, 2025

Abstract

This thesis addresses the challenge of achieving quality 3D scene reconstructions from only RGB input. Specifically, the purpose of this project is to eliminate the depth input requirement of GauSTAR, which is a method for reconstructing and tracking topology-changing 3D surfaces in dynamic scenes using a combination of meshes and 3D Gaussians. Its current framework relies on depth sensor captures to estimate scene flow, limiting its use to only the problem setups with available depth images. The approach detailed in this thesis proposes to replace the depth-dependent scene flow estimation in the mesh warping module of GauSTAR with a two-step coarse-to-fine scene flow estimation algorithm that solely uses RGB images. The algorithm consists of an initial multi-view triangulation to estimate 3D motion, followed by per-vertex nonlinear refinement using Gauss–Newton optimization with Levenberg–Marquardt damping. Experimental results of the algorithm show that, although the RGB-only estimation method exhibits slightly higher endpoint error metrics compared to depth-informed GauSTAR, it provides visually plausible scene flow estimations. These results establish a promising direction for depth-free dynamic scene reconstruction and provide a solid foundation for further research into fully RGB-based methods within the GauSTAR framework.

Acknowledgements

This work was carried out using the original GauSTAR codebase, which was provided by my advisors. They have supported me throughout the development of this thesis.

Contents

1	Introduction	1
1.1	Focus of this Work	2
1.2	Thesis Organization	3
2	Related Work	5
2.1	Optical Flow Estimation	5
2.2	Scene Flow Estimation	5
2.2.1	Two-Stage Scene-Flow Pipelines	6
3	Materials and Methods	7
3.1	GauSTAR Framework Overview	7
3.2	Scene Flow Estimation without Depth Overview	7
3.3	Multi-View Triangulation	8
3.4	Per-Vertex Nonlinear Refinement	9
3.5	Post Processing and Integration	10
4	Experiments and Results	11
4.1	Implementation Details	11
4.2	Scene Flow Prediction	11
4.3	Results	15
4.4	GauSTAR integration	18
5	Discussion	21
5.1	Implications	21
5.2	Limitations & Future Work	21
A	The First Appendix	23
A.1	Source Code for Scene Flow Estimation and Mesh Warping	23

List of Figures

3.1	Scene flow estimation pipeline	8
4.1	Scene flow estimation for frames 11 and 18 on 'passing ball' track with disabled topology changes	11
4.2	Optimistic GauSTAR result with proposed method and topology updates turned off, frame 11 on 'passing ball' track	18
4.3	Frame 24 in 'Vest' Track GauSTAR results for original and proposed mesh warping modules.	19

List of Tables

4.1	Scene-flow visualizations	12
4.2	Error-vs-ground-truth visualizations (GauSTAR, initial, refined).	15
4.3	Mean scene flow endpoint errors (EPE) in meters for different frames.	18

Chapter 1

Introduction

Vision is a fundamental tool for humans to make sense of the surrounding environment. Consequently, many technologies rely on the ability to visually represent the outside world and interpret this representation. Today, the ability to reconstruct 3D scenes accurately is essential for modern computer vision applications in a great variety of fields. It enables the generation of photorealistic novel views which is important for AR/VR applications as well as the movie and game industries. In robotics, reliable 3D representations are fundamental for intelligent systems that need to interpret 3D scenes to be able to navigate and safely interact with the world.

One crucial consideration in developing these vision-dependent systems is that the world is a dynamic environment. The scene changes with time, which creates a need to track surfaces over time to preserve the temporal consistency. The scenes can contain non-rigidly moving objects or *topology changes* such as splitting, merging, and deforming surfaces. However, the non-rigidity of motion creates a severely under-constrained problem and many of the recent methods rely on static assumptions. Static scene representation has seen a considerable progress lately with competitively efficient and photo realistic rendering results. However, extending these results to dynamic scenes is still an ongoing problem. Methods that provide high-quality, accurate reconstructions while also consistently tracking the surfaces are therefore required for more "realistic" dynamic scene reconstructions.

In the past years, neural scene representations have revolutionized the field by encoding scene appearance and geometry in different learnable models. Neural Radiance Fields (NeRF) [9] became a significant implicit representation method for generating detailed scenes. However, despite its high-quality renderings NeRF tends to be slow to train or render as a result of its costly ray sampling and it can struggle with precise surface extraction due to density-geometry ambiguity. Follow-up works like NeuS [13] addressed the latter issue by integrating signed distance functions (SDFs) into radiance fields, enabling surface extraction via Marching Cubes . However, the obtained surfaces still lack consistent tracking unless underlying templates are used.

Explicit point and surfel representations have re-emerged as powerful competitors to NeRF variants. Point-based radiance fields (e.g. Neural Point Cloud rendering) and surfels (or oriented disks) directly store scene information at discrete locations. A breakthrough explicit method is 3D Gaussian Splatting (3DGS)[4]. It represents a scene with a set of anisotropic 3D Gaussians. Each Gaussian is a tiny ellipsoid in space with color and opacity. The Gaussians preserve the continuous nature of volumetric fields but avoid the costly sampling approach of NeRF[9]. The explicit 3DGS approach matches the rendering quality of NeRF while achieving more efficient, close to real-time rendering. The explicit nature of this method, combined with its efficiency facilitates tracking over time and various methods were derived from 3DGS to extend its use to dynamic scenes. However, accurate reconstructions that balance tracking capability with flexibility to represent topology changes remained an unsolved challenge.

In response to these challenges, GauSTAR was proposed as a method capable of reconstructing photo-realistic appearances and accurate surface geometries with consistent tracking as topology changes across frames [15]. It relies on multi-view RGBD information and meshes with bound Gaussians to generate Gaussian Surfaces. GauSTAR introduces adaptive Gaussian unbinding and surface re-meshing to adapt the frame in light of topology changes. Gaussian unbinding enables previously attached Gaussians to detach from their mesh faces in regions of high deformation or reconstruction error, allowing them to move freely and represent emerging surfaces. These unbound Gaussians then guide the surface re-meshing process, where new geometry is reconstructed and smoothly integrated into the existing mesh. Depth information is critical for both stages. It is used both to detect topology changes via scene flow and to fuse new surfaces during re-meshing. To ensure temporal consistency, GauSTAR leverages a surface-based scene flow method that back-projects 2D optical flow into 3D using depth data, providing a robust initialization for frame-by-frame tracking even under large or rapid motion in 3D. [15]

1.1 Focus of this Work

The current framework of GauSTAR relies heavily on the depth input from IR sensors and sophisticated geometry processing software to estimate scene flow. The heavy input requirements restrict GauSTAR’s applicability in general public scenarios. This thesis aims to extend GauSTAR to work on RGB data only and improve its performance on geometry details. To overcome the reliance on depth data, this project proposes a pipeline that estimates 3D scene flow using only RGB images and known camera parameters. The core idea is to track the motion of points across views by leveraging optical flow between temporally adjacent frames for each camera. For this purpose, a two-step coarse-to-fine scene flow estimation algorithm is proposed. First, the system computes forward and backward optical flow fields for each RGB view. Next, instead of relying on depth to lift these pixel motions into 3D, the method casts rays from each camera through the target pixels in the next frame, using the camera intrinsics and extrinsics. Since the viewing directions of the cameras do not align with the ideal stereo setup, the true location of the point in 3D space can be estimated by solving a least-squares problem that finds the best intersection point of these rays. This solution is not accurate enough, but it forms a good starting point for the following per-vertex refinement step. The proposed refinement algorithm optimizes each vertex position independently using a Gauss-Newton step with Levenberg–Marquardt damping to minimize total squared reprojection error between the observed and predicted 2D flows. In effect, the pipeline triangulates the new 3D position of each mesh vertex using epipolar geometry and optical flow as the primary cues.

The algorithm was first evaluated on the initial coarse meshes produced during the GauSTAR preprocessing stage, prior to any depth-based optimization. In this setup, the full GauSTAR pipeline was bypassed, and only the RGB-based warping method was applied to assess its standalone performance. Subsequently, the algorithm was integrated into the full GauSTAR pipeline by replacing the depth-dependent mesh warping module. In this integrated version, all other stages—such as aggregation across views, outlier rejection, smoothing, and propagation—remained consistent with the original implementation.

The approach proposed in this thesis can be seen as the initial steps for achieving a robust, depth-free scene flow estimation within the original GauSTAR pipeline. While the method demonstrates promising results and establishes a functional proof of concept, it does not yet achieve the full level of robustness or seamless integration required for production use. This limitation is primarily due to the time constraints of the project, which restricted further iteration and optimization. Nevertheless, this work presents a strong foundation and a viable direction for future endeavors of developing a new GauSTAR-based method with RGB-only tracking. Within the scope of this thesis, several alternative strategies were explored and experimentally evaluated, reinforcing the value of this direction. With further improvement, particularly concerning the integration of the proposed algorithm into GauSTAR’s existing pipeline, the method has the potential

to offer a practical and scalable solution for real-world applications where depth data is unavailable.

1.2 Thesis Organization

This thesis is structured as follows. Chapter 2 reviews related work in optical flow and scene flow estimation methods, particularly in the context of dynamic 3D reconstruction using explicit representations. Two-step scene flow estimation methods that inspired the proposed approach are also highlighted. Chapter 3 introduces the proposed RGB-only scene flow estimation pipeline, detailing its components, including multi-view triangulation, nonlinear refinement, and current integration with the GauSTAR framework. Chapter 4 presents experimental results and visualizations that evaluate the method both in isolation and when integrated into GauSTAR. Chapter 5 discusses the implications, limitations, and potential future directions of this work. Finally, the Appendix provides extended numerical results supporting the evaluations as well as the source code for the proposed module.

Chapter 2

Related Work

Dynamic 3D scene reconstruction aims to recover scene appearance and geometry over time from visual data, often using neural representations. Recent neural representations such as NeRF[9] and 3DGS[4] have significantly advanced the static reconstruction field, while their extensions to dynamic scenes have led the way to strong methods in dynamic scene reconstruction. 3DGS variants have shone in this field since their explicit representation facilitates tracking. Methods like GauSTAR[15] combine Gaussians with mesh structure and depth-based flow to handle arbitrary deformations and topology changes, whereas fully RGB-driven approaches like Dynamic 3D Gaussians [7] and SC-GS [3] forego depth and templates by relying on persistent Gaussians and learned motion fields. The trade-off lies in robustness versus dependence on extra sensors: GauSTAR’s depth-informed warping yields reliable tracking but uses RGB-D input, while purely image-based methods avoid depth but must carefully regularize motions to achieve coherent tracking. This thesis builds on these foundations, aiming to integrate the best of both worlds: leveraging only RGB videos (like SC-GS or Dynamic 3D Gaussians) while preserving GauSTAR’s strength in handling complex motions and topology changes through scene-flow estimation.

2.1 Optical Flow Estimation

Optical flow, which is the per-pixel 2D motion between video frames, is a critical building block for dynamic 3D reconstruction. One important work in this field that is also an essential element in GauSTAR’s bidirectional optical flow estimation is RAFT (Recurrent All-Pairs Field Transforms)[11]. RAFT builds a multi-scale 4D correlation volume for all pairs of pixels between two images and then iteratively updates the flow field via a recurrent unit that looks up correlations at multiple scales.

For dynamic radiance fields, optical flow can serve as an intermediate supervision or prior for 3D motion. GaussianFlow [2] and MotionGS [16], two recent works that connect 2D flow with Gaussian Splatting, are important inspirations for this thesis. The approach in GaussianFlow is to differentiably project the dynamic 3D Gaussians into the image plane and measure their motion in 2D, effectively obtaining a flow field induced by the Gaussians. MotionGS stabilizes the Gaussian trajectories and captures object deformations more reliably by explicitly injecting optical flow knowledge. The success of these approaches suggests that learned optical flow can replace depth images to inform where each point in the scene moves, for tasks that aim for accurate and topologically consistent dynamic scene models.

2.2 Scene Flow Estimation

While optical flow describes 2D image motion, scene flow generalizes this to 3D: it is the vector field describing how every point in the 3D scene moves between two time steps. Scene flow is an essential part of

the GauSTAR pipeline as it is used to update the mesh vertex positions. However, it is also the main cause of depth dependency in GauSTAR. Neural SDF Flow [8] represents a promising depth-free approach to link dynamic scene reconstruction with scene flow. A related method, Neural Scene Flow Fields (NSFF)[6], integrates a continuous 3D motion field into the NeRF framework, enabling temporally consistent view synthesis for dynamic scenes. By jointly optimizing both appearance and scene flow, NSFF improves the fidelity of reconstructions across time, although it assumes a fixed scene topology and does not account for new object insertions.

2.2.1 Two-Stage Scene-Flow Pipelines

A number of recent scene flow methods follow a two-stage design pattern similar to the one adopted in this thesis: a coarse initialization followed by a localized refinement step. This design is motivated by the need for robustness in the presence of noise and occlusion, while still achieving high-precision alignment. For instance, SceneFlowFields++ [10] first extracts sparse, reliable correspondences using stereo and optical flow, and then performs dense interpolation via a variational solver to refine the flow. Similarly, deep learning-based models like PointPWC-Net [14] and FlowStep3D [5] produce a coarse 3D flow estimate using feature correlation or cost volumes, and then apply residual updates at progressively finer scales. These updates act as learned analogues of Gauss–Newton steps, correcting errors around an already reasonable initial guess.

Other methods such as NRPCR-SF [1] and EulerFlow [12] follow the same philosophy in the context of non-rigid registration and continuous motion modeling. NRPCR-SF first predicts flow and occlusion jointly and then propagates flow into hard-to-estimate regions using learned neighborhood interpolation. EulerFlow fits a global neural ODE to model scene motion over time, then refines it iteratively through self-supervised optimization, resembling a global variant of Gauss–Newton. These methods confirm the effectiveness of separating scene flow into a robust coarse estimation phase and a fine-scale correction phase. The refinement module in the pipeline presented in this thesis, based on per-vertex Gauss–Newton optimization with Levenberg–Marquardt damping, follows this same structure, but operates explicitly in geometric space, refining triangulated positions using only multi-view image alignment via optical flow. This explicit, interpretable structure has potential for both accuracy and reliability, especially in the absence of depth sensors.

Chapter 3

Materials and Methods

3.1 GauSTAR Framework Overview

GauSTAR is a method for reconstructing and tracking deforming 3D surfaces in dynamic scenes using a combination of meshes and 3D Gaussians. The core idea is to attach Gaussians that encode both geometry and appearance to the faces of a mesh, allowing for photorealistic rendering and precise surface tracking. These "Gaussian Surfaces" evolve over time to follow object motion and adapt when the object changes shape or splits into parts.

The process starts with multi-view RGB-D videos. From one frame to the next, GauSTAR uses scene flow estimated from optical flow and depth to initialize how surfaces are expected to move. This gives a good starting guess for how objects deform. Using this initialization, the system optimizes both the mesh vertex positions and the parameters of the Gaussians (such as position, shape, color, and orientation) to match the appearance and geometry seen from all camera views.

To handle topology changes, GauSTAR detects which regions no longer match their original mesh structure. In these areas, it allows the Gaussians to "unbind" from the mesh and move independently. These freed Gaussians are then used to create new mesh regions through surface re-meshing, ensuring the representation remains accurate and consistent as the object evolves.

This combination of bound and unbound Gaussians allows GauSTAR to flexibly track known surfaces and discover new ones. The method avoids relying on predefined templates and instead adapts to the actual dynamics of the scene, making it especially suited for real-world applications like motion capture, scene editing, and augmented reality.

3.2 Scene Flow Estimation without Depth Overview

In the original GauSTAR pipeline, mesh warping between consecutive frames is performed using the bi-directional optical flow obtained with RAFT in conjunction with the depth images of the corresponding frames. The GauSTAR mesh warping algorithm can be summarized as follows. A 3D vertex on the mesh is projected into each camera's image plane using the known camera parameters. The forward optical flow at the projected pixel is retrieved from the bi-directional optical flow estimation of the frame. The optical flow value of a certain pixel indicates the expected image-space motion in the next frame. The pixel is moved according to optical flow and the next frame position is obtained. Sampling the depth image of the next frame, the depth value at the moved pixel is read. The new pixel and its depth value are re-projected into 3D space, recovering the new estimated position of the vertex. The difference between the original and new 3D positions gives the displacement vector, or scene flow for the vertex. This process is repeated across all camera views and followed by a filtering and aggregation stage.

The proposed method replaces the depth based scene-flow estimation with a two-step, triangulation-based approach. After the pixel is moved based on the forward optical flow, a 3D ray is cast from the camera center through the new pixel position using camera parameters. Since the depth information is not directly available, it needs to be estimated using multi-view triangulation. However, the casted rays do not always intersect since the baselines of the cameras do not align perfectly, as in the case of an ideal stereo setup. Therefore, the best-fit intersection point is calculated via least-squares minimization, estimating the new 3D position of the vertex. This calculation does not provide satisfactory sensitivity, especially for high-frequency regions, and tends to underestimate the movement. In the proposed algorithm, the triangulation results are used as a reliable initial guess for the following per-vertex nonlinear refinement step. This part takes the coarse estimate and returns a finer one. It does this by minimizing the difference between where the vertex currently projects and where the flow says it should appear. The update is computed using a damped Gauss–Newton optimization, which makes small, stable corrections to improve accuracy. This refinement step significantly improves the accuracy of motion estimation in regions with fine-scale geometry, such as hands and clothing folds, which are particularly sensitive to small localization errors.

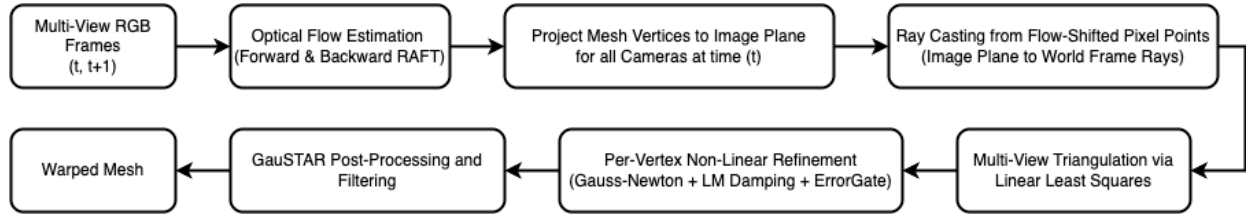


Figure 3.1: Scene flow estimation pipeline

3.3 Multi-View Triangulation

For each vertex on the mesh, the 3D coordinates are projected onto the image plane of each camera using the known intrinsic (K) and extrinsic ($[R|t]$) parameters. The corresponding pixel in image space is used to query the forward optical flow, which indicates how that pixel moves in the next frame. Let $\mathbf{p} \in \mathbb{R}^2$ denote the original pixel location, and $\mathbf{f} \in \mathbb{R}^2$ denote the optical flow vector. The new pixel location becomes:

$$\mathbf{p}' = \mathbf{p} + \mathbf{f} \quad (3.1)$$

Since the depth information in world frame is not directly available, it is not possible to directly recover the 3D point. The solution is to cast a ray from the camera center through the pixel position $\mathbf{p}'$ in the image plane. This is achieved by computing a ray direction vector $\mathbf{d}_{\text{cam}}$ in the local camera coordinate system.

$$\mathbf{d}_{\text{cam}} = \begin{bmatrix} \frac{u' - c_x}{f_x} \\ \frac{v' - c_y}{f_y} \\ 1 \end{bmatrix} \quad (3.2)$$

Where (u', v') are the pixel coordinates of point $\mathbf{p}'$, (c_x, c_y) are the principal point offsets and f_x, f_y are the focal lengths in x and y directions. This ray is then transformed into world coordinates using the inverse of the camera rotation matrix R . The world coordinate direction vector is then normalized to obtain a unit vector.

$$\mathbf{d}_{\text{world}} = R^\top \cdot \mathbf{d}_{\text{cam}} \quad (3.3)$$

$$\mathbf{d}'_{\text{world}} = \frac{\mathbf{d}_{\text{world}}}{\|\mathbf{d}_{\text{world}}\|} \quad (3.4)$$

The full ray in world coordinates is defined as below. It describes a ray starting from the camera center $\mathbf{C}$ in world coordinates and extending with unit steps scaled by parameter t in the direction $\mathbf{d}'_{\text{world}}$.

$$\mathbf{r}(t) = \mathbf{C} + t \cdot \mathbf{d}'_{\text{world}} \quad (3.5)$$

In the traditional stereo depth estimation setup, the rays cast from two cameras through the pixels corresponding to the same 3D point are expected to intersect, and this intersection point indicates the 3D location. This intersection of rays occurs in the ideal case where camera baselines are aligned and measurements are not noisy. In a realistic scenario with multiple cameras of highly varying view directions, the rays are not expected to intersect. Therefore, it is necessary to pick the best-fit 3D point through solving a least-squares problem. For a set of valid rays $\{(\mathbf{C}_i, \mathbf{d}_i)\}$, the goal is to find a point $\mathbf{x} \in \mathbb{R}^3$ that minimizes the total squared distance to all rays, which can be expressed as follows.

$$\sum_i \left\| \left(\mathbf{I} - \mathbf{d}_i \mathbf{d}_i^\top \right) (\mathbf{x} - \mathbf{C}_i) \right\|^2 \quad (3.6)$$

This leads to the normal equation below whose solution gives the estimated 3D position $\mathbf{x}$ for the vertex in the next frame.

$$\left(\sum_i \left(\mathbf{I} - \mathbf{d}_i \mathbf{d}_i^\top \right) \right) \mathbf{x} = \sum_i \left(\mathbf{I} - \mathbf{d}_i \mathbf{d}_i^\top \right) \mathbf{C}_i \quad (3.7)$$

3.4 Per-Vertex Nonlinear Refinement

Due to several factors like occlusion, noise, or imprecise calibration, the initial estimate provided by the multi-view triangulation can be inaccurate. To improve the estimation of vertex positions, a nonlinear step carrying out per-vertex bundle adjustment refinement is introduced. This refinement optimizes the 3D position of each vertex to better match all of the constraints, like the observed 2D flow correspondences across multiple views, while regularizing the solution to prevent overfitting or divergence in ill-posed cases.

The refinement step has the form of a nonlinear least squares problem. The main goal is to obtain a refined 3D position $\mathbf{x} \in \mathbb{R}^3$ for each vertex, such that its projection to the image plane matches the movement predicted by the optical flow for that vertex. At the same time, the vertex has to be visible in enough camera views to be considered for refinement. For $\mathbf{p}_i$ being the projection of $\mathbf{x}$ into the image plane of camera i , and $\mathbf{f}_i$ being the forward optical flow vector at that pixel. The target pixel location is $\mathbf{p}_i + \mathbf{f}_i$. The residual, or the error vector, for this camera is given by the image-space difference between projected refined vertex location and the predicted flow position.

$$\mathbf{r}_i(\mathbf{x}) = \Pi_i(\mathbf{x}) - (\Pi_i(\mathbf{x}_0) + \mathbf{f}_i) \quad (3.8)$$

where $\Pi_i(\cdot)$ denotes the perspective projection function using the intrinsic matrix K_i and extrinsic matrix $[R_i | t_i]$ for camera i , and $\mathbf{x}_0$ is the initial triangulated position of the vertex.

The overall objective is to minimize the sum of squared errors over all cameras where the optical flow is valid:

$$\min_{\mathbf{x}} \sum_i |\mathbf{r}_i(\mathbf{x})|^2 \quad (3.9)$$

The posed least squares problem is solved using a single iteration of the Gauss–Newton method. This method approximates the solution by linearizing the residuals around the current estimate $\mathbf{x}_k$ and solving the following normal equations:

$$\left(\sum_i \mathbf{J}_i^\top \mathbf{J}_i \right) \Delta \mathbf{x} = - \sum_i \mathbf{J}_i^\top \mathbf{r}_i \quad (3.10)$$

where $\mathbf{J}_i = \partial \mathbf{r}_i / \partial \mathbf{x}$ is the Jacobian of the residual with respect to the 3D vertex position. The refined position is updated as $\mathbf{x}_{k+1} = \mathbf{x}_k + \Delta \mathbf{x}$.

To achieve a robust estimate, especially in cases with low visibility (vertex seen by few cameras) or ill-conditioned geometry, a Levenberg–Marquardt damping term is introduced. This leads to a regularized solution of the form:

$$\left(\sum_i \mathbf{J}_i^\top \mathbf{J}_i + \lambda \mathbf{I} \right) \Delta \mathbf{x} = - \sum_i \mathbf{J}_i^\top \mathbf{r}_i \quad (3.11)$$

where λ is a small positive damping constant. This regularization term becomes crucial for cases when the Hessian matrix is near singular. It encourages the solution to be more conservative and prevents the update steps $\Delta \mathbf{x}$ from becoming excessively large.

As an additional measure, the step size $|\Delta \mathbf{x}|$ is clamped to a maximum threshold to prevent vertices from diverging to invalid locations due to outlier flows. This threshold was experimentally determined for the tested sequence. If the computed update exceeds this threshold or the solution becomes numerically unstable, the vertex is left unchanged, and the estimate falls back to the triangulated initial solution.

Furthermore, a reprojection-error gate is applied to validate each candidate update. Specifically, the total reprojection error across all valid views is computed both before and after applying the update. If the updated vertex position results in a higher total reprojection error (i.e., the sum of 2D residual magnitudes increases), the update is rejected. This serves as a conservative safeguard against harmful updates in the presence of inconsistent flow or geometric degeneracies. As a result, only updates that demonstrably improve the alignment with observed flows are accepted.

3.5 Post Processing and Integration

In accordance with the GauSTAR pipeline, the triangulation stage was followed by a series of post-processing steps. These procedures are essential to handle noise, enforce spatial coherence, and integrate the estimated motion into the final warped mesh representation. At the outlier rejection stage, a z-score-based filtering is applied to discard outlier vectors whose magnitude deviates significantly from the mean. This stage is crucial to eliminate spurious results. Then, to ensure the reliability of motion estimation, a visibility threshold is enforced. Vertices must be observed by a minimum number of valid camera views to be considered in the update. Following this, a spatial propagation on the mesh is carried out to iteratively fill in invalid or untrusted displacement values by averaging over neighboring vertices. Lastly, a color smoothing step was applied to enforce local continuity. The final updated mesh is obtained by applying the smoothed displacement vectors to the original vertex positions. These steps were not optimized in the context of this thesis.

Chapter 4

Experiments and Results

4.1 Implementation Details

The method was tested on the same RGB data as GauSTAR setup. RGB video frames captured by 52 RGB cameras in the designated capture studio were used for all test and development stages. Sequences were captured at a resolution of 3004×4092 and 30 fps. Since the goal of this project is to eliminate depth input requirements of GauSTAR, the code was designed to replace only the depth-dependent mesh warping module. Other parts of GauSTAR pipeline were preserved, and the new algorithm was tested separately and as part of the full pipeline.

Mesh warping with triangulation algorithm is an initial step towards the goal of eliminating the depth input requirement of GauSTAR. It was therefore developed and tested to perform similarly to GauSTAR, primarily on non-topology-changing cases. For this purpose, only static parts of the sequences were tested. The full GauSTAR pipeline was run with mesh topology updates turned off.

4.2 Scene Flow Prediction

The algorithm was tested on different simple sequences and the estimation accuracy was observed. In the below figure 4.1, the scene flow estimation results via triangulation were demonstrated for only two frames. Each frame was scaled within itself for the color mapping.

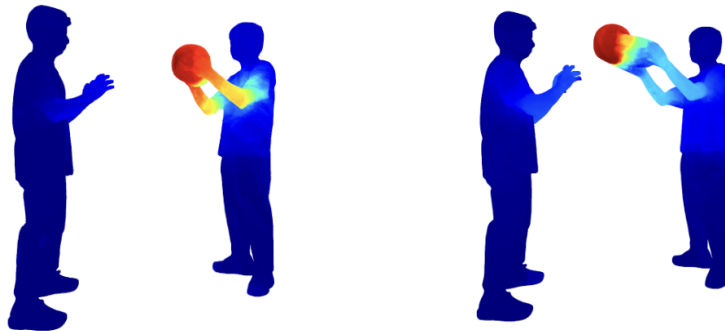


Figure 4.1: Scene flow estimation for frames 11 and 18 on 'passing ball' track with disabled topology changes

Scene flow prediction for a sequence of frames was calculated as a point cloud and visualized for the two proposed steps, along with the ground truth scene flow from GauSTAR. For visualisation of scene flow vectors, color maps with the scale of ground truth scene flow range per frame were preferred. Results for the 'Vest' track and the used scales are listed in the table 4.1 below.

Table 4.1: Scene-flow visualizations

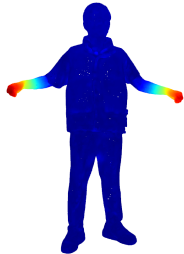
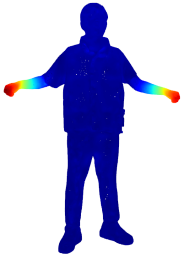
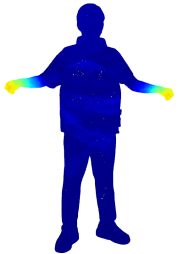
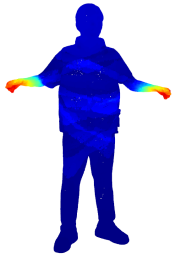
Frame	GT	GauSTAR	Initial Flow	Refined Flow	Scale
21					0.000027– 0.066567
31					0.000014– 0.040156
41					0.000009– 0.007618

Table 4.1 (continued)

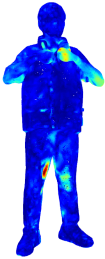
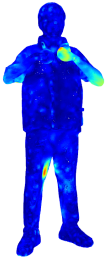
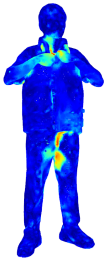
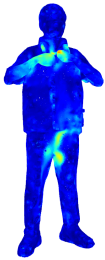
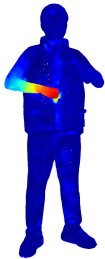
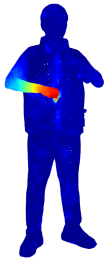
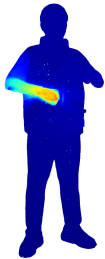
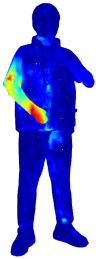
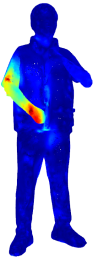
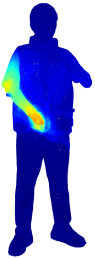
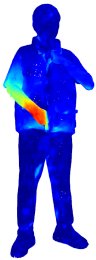
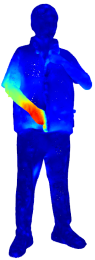
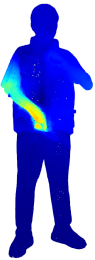
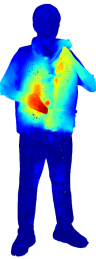
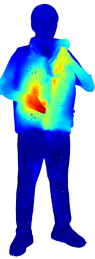
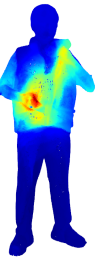
Frame	GT	GauSTAR	Initial Flow	Refined Flow	Scale
51					0.000021– 0.004869
61					0.000009– 0.005298
71					0.000010– 0.012301
81					0.000011– 0.020038

Table 4.1 (continued)

Frame	GT	GauSTAR	Initial Flow	Refined Flow	Scale
91					0.000009– 0.014184
101					0.000010– 0.012935
111					0.000011– 0.008809
121					0.000020– 0.022971

4.3 Results

The table 4.2 below visualizes the errors in the scene flow estimation above. The error visuals were rendered from point clouds.

Table 4.2: Error-vs-ground-truth visualizations (GauSTAR, initial, refined).

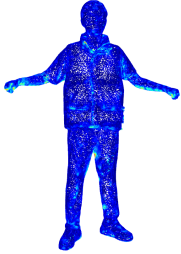
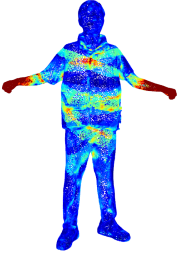
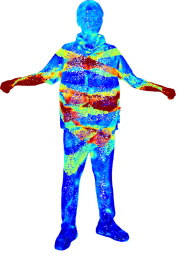
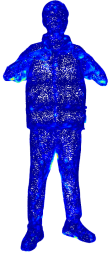
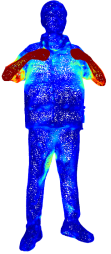
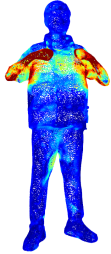
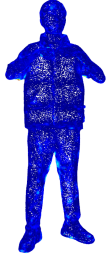
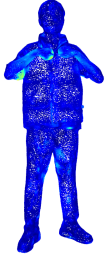
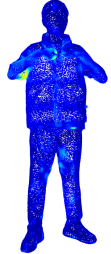
Frame	GauSTAR vs GT	Initial vs GT	Refined vs GT	Scale
21				0.000010– 0.006071
31				0.000005– 0.008976
41				0.000004– 0.009248
51				0.000005– 0.008087

Table 4.2 (continued)

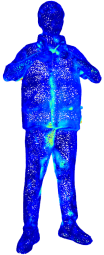
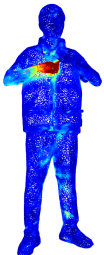
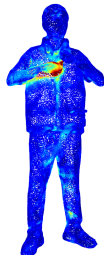
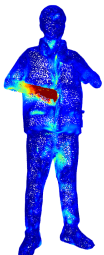
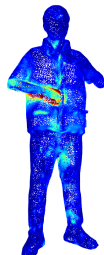
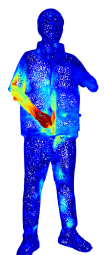
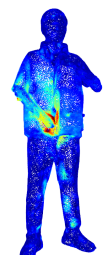
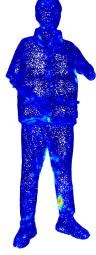
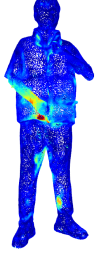
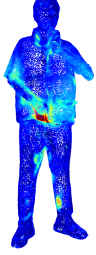
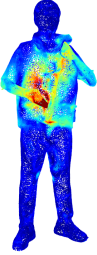
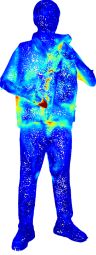
Frame	GauSTAR vs GT	Initial vs GT	Refined vs GT	Scale
61				0.000010– 0.006731
71				0.000006– 0.006481
81				0.000007– 0.007453
91				0.000008– 0.008065

Table 4.2 (continued)

Frame	GauSTAR vs GT	Initial vs GT	Refined vs GT	Scale
101				0.000012– 0.007143
111				0.000009– 0.010757
121				0.000004– 0.010861

For the numerical evaluation of the scene flow estimates, the mean end-point-error (MEPE), which is the average Euclidean distance between predicted and ground truth scene flow vectors of GauSTAR, was returned. The results show, as expected, that predicted scene flow performs worse than the depth-informed scene flow calculation of GauSTAR. The MEPE difference between predicted and ground truth ranges from 0.5mm to 0.5cm. Even though the visual results of the refinement step seem promising, the MEPE turns out to be the same or slightly worse than the triangulated result, with 0.1 to 5 mm of increase. This might be due to a lack of smoothness, a lack of global constraints, and overfitting. These possibilities were briefly considered but not dealt with due to the time restrictions of the project.

The table 4.3 below shows the error logs corresponding to several frames as well as the mean for the MEPE errors in the corresponding categories across all frames (21-183). GauSTAR's error is taken as a reference or the ground truth in this case. Coarse Error is the error after coarse scene flow estimation and Refined Error is the error after the refinement step.

Frame	GauSTAR	Coarse Error	Refined Error
24	0.0006	0.0025	0.0027
34	0.0006	0.0018	0.0017
54	0.0005	0.0008	0.0008
74	0.0005	0.0011	0.0011
84	0.0005	0.0012	0.0012
⋮	⋮	⋮	⋮
Mean	0.000535	0.001340	0.001456

Table 4.3: Mean scene flow endpoint errors (EPE) in meters for different frames.

4.4 GauSTAR integration

As indicated in the table 4.3 and explained above, the scene flow estimation errors are higher than GauSTAR's. The refinement stage needs to be improved to provide better smoothing and robustness against noise in the optical flow estimation, which is thought to be the main cause of the lack of improvement in error metrics. Additionally, the optimal integration of the developed method to the GauSTAR pipeline is still an unresolved issue. GauSTAR's existing optimization methods need to be tuned or new ones need to be introduced to enable more meaningful results. Below is a sample result from a run of GauSTAR where its mesh warping module was replaced with the proposed method. The sample is from one of the earliest frames of the sequence; therefore, the motion is quite small. Despite this, there are visible artifacts on the generated mesh, especially when zoomed in. The magnified sample result in the right in 4.2 demonstrates some of these artifacts in the elbow and fingers of the subject. This is due to the fact that the integration of the proposed method into GauSTAR was not optimal, and it ran on previous configurations. The smoothing, outlier rejection, and color refinement post-processing steps of GauSTAR are crucial for the photorealistic end result. These stages should be optimized for the new method to achieve robust reconstruction results. Figure 4.3 demonstrates some results from the 'Vest' track. This shows a more visible error with the hands of the figure, which have the main movement, missing parts and texture information.

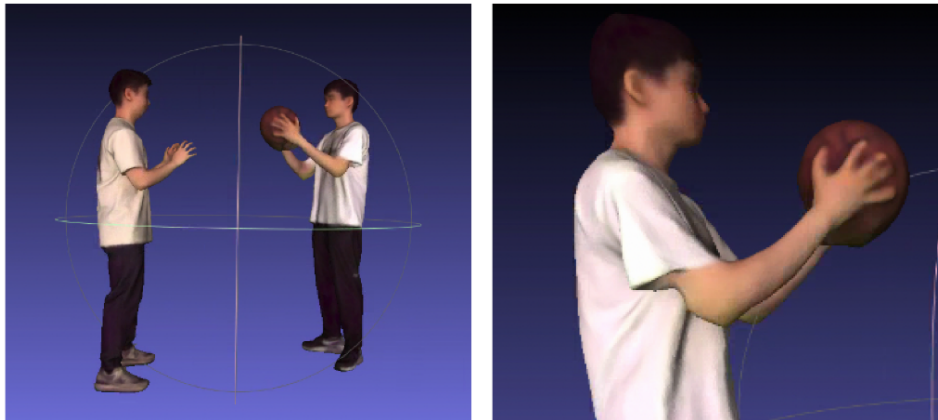


Figure 4.2: Optimistic GauSTAR result with proposed method and topology updates turned off, frame 11 on 'passing ball' track

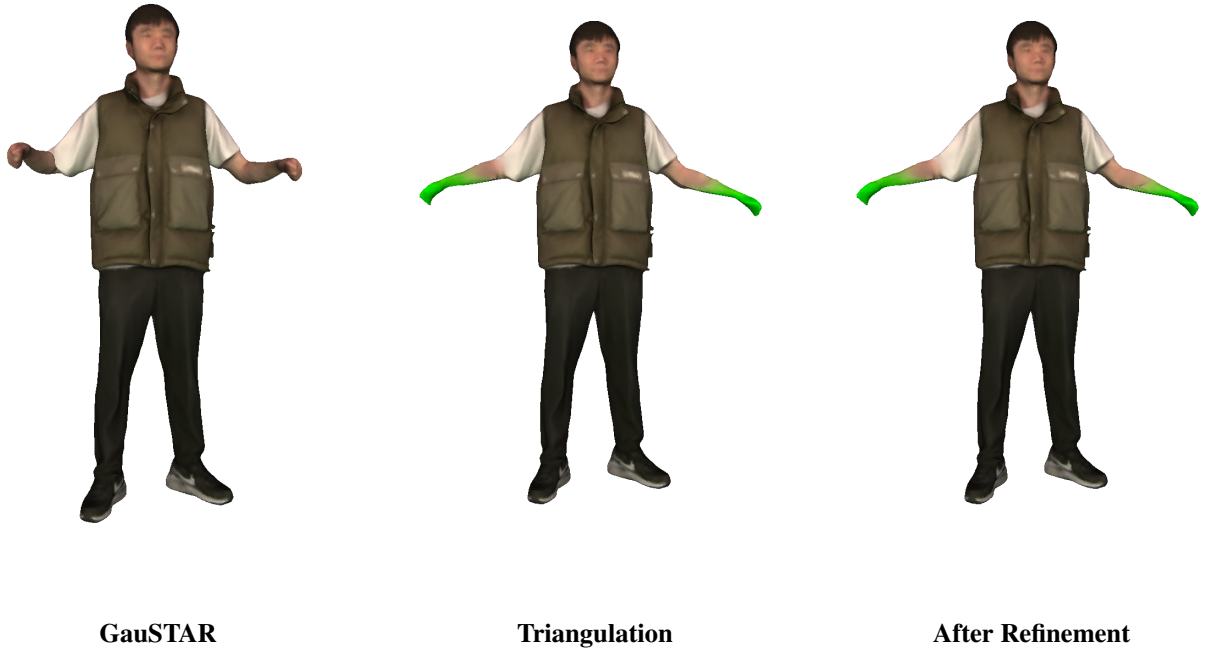


Figure 4.3: Frame 24 in 'Vest' Track GauSTAR results for original and proposed mesh warping modules.

Chapter 5

Discussion

5.1 Implications

The results of this thesis demonstrate that scene flow estimation entirely based on RGB inputs is a feasible and promising alternative to approaches such as the original GauSTAR that depend on depth sensor inputs. The presented scene flow estimation pipeline, which is composed of a coarse triangulation step followed by a per-vertex nonlinear refinement, aligns with the two-stage structure found in recent works, such as SceneFlowFields++[10] and FlowStep3D [5]. These methods show that separating coarse initialization and localized refinement yields more robust estimations in the presence of noise, occlusion, or imperfect camera geometry. Unlike the mentioned end-to-end learned models, the proposed pipeline uses analytical Gauss-Newton optimization guided by multi-view optical flow. This approach improves interpretability and makes integration into the explicit surface-based GauSTAR system more tractable.

Although the refinement step did not consistently improve quantitative metrics such as Mean End Point Error (MEPE), it succeeded in improving visualized results and recovering finer details in high-frequency regions. This gain shows the method is a promising foundation and it can return quality results after the improvement of the refinement and GauSTAR post-processing stages.

5.2 Limitations & Future Work

Despite the promising outcomes, the current method falls short of the robustness and seamless integration required for full GauSTAR replacement. As results show, the refinement step occasionally increases the overall error, likely due to a combination of noisy optical flow, lack of global smoothness priors, and weak constraints in regions with limited visibility. These issues could be addressed by incorporating learned priors, spatiotemporal regularization, or more expressive deformation models similar to those used in EulerFlow[12] or Neural Scene Flow Fields [6].

Furthermore, this method was developed without regard for the topology-changing cases, even though this is the main focus of GauSTAR. This project aims to develop the first stages of a depth-input-free GauSTAR variant. Therefore, creating a working algorithm that matches the existing structure was prioritized in this project. Several other candidate methods were explored during development, including learning-based and hybrid approaches, but the chosen solution offered the clearest trade-off between complexity and performance under the given time constraints.

Future directions include extending the current method with global regularization across the mesh, experimenting with learned confidence weights for flow reliability, and building differentiable modules that allow end-to-end fine-tuning within the GauSTAR pipeline. Moreover, incorporating motion priors from recent dynamic Gaussian models like MotionGS[16] or SC-GS[3] could further increase the robustness of the

scene flow estimation without compromising the modular and interpretable nature of the current approach.

Appendix A

The First Appendix

A.1 Source Code for Scene Flow Estimation and Mesh Warping

```
1  import os
2  import numpy as np
3  import matplotlib.pyplot as plt
4  from setuptools.sandbox import save_path
5  from skimage.util import view_as_windows
6  import torch
7  import torch.nn.functional as F
8  import trimesh
9  import cv2
10 import tiff file as tf
11 from tqdm import tqdm
12 import open3d as o3d
13 import time
14 import pandas as pd
15 from scipy.optimize import least_squares
16
17
18 """
19 code to predict scene flow from RGB input and initial mesh vertices
20 """
21
22 def linear_triangulation(rays_dir, rays_valid, C_orig, vert_num, eps=1e-8):
23     """
24     Perform linear triangulation to find 3D vertex positions from multiple
25     ↪ camera rays.
26
27     This function solves for vertex positions that minimize the distance to
28     ↪ all
29     viewing rays using a least-squares approach.
30
31     Args:
32         rays_dir: np.ndarray, shape (C, vert_num, 3) - normalized ray
33         ↪ directions from each camera
34         rays_valid: np.ndarray, shape (C, vert_num) - boolean mask of valid
35         ↪ rays
```

```
32     C_orig:      np.ndarray, shape (C, 3) - camera centers in world
                 ↪ coordinates
33     vert_num:    int - number of vertices to triangulate
34     eps:         float - regularization parameter for numerical stability
35
36     Returns:
37     Xnxt:        np.ndarray, shape (vert_num, 3) - triangulated vertex
                 ↪ positions
38     """
39     # number of cameras
40     C = rays_dir.shape[0]
41     # Initialize accumulation matrices
42     AtA = np.zeros((vert_num, 3, 3), dtype=np.float32)
43     Atb = np.zeros((vert_num, 3), dtype=np.float32)
44     I3 = np.eye(3, dtype=np.float32)
45     for cam_idx in range(C):
46         d = rays_dir[cam_idx]
47         ok = rays_valid[cam_idx]
48         #projection matrix
49         A_c = I3[None] - d[:, :, None] * d[:, None, :]
50         A_c[~ok] = 0 # Zero out invalid rays
51         # Accumulate normal equations
52         AtA += A_c
53         Atb += (A_c @ C_orig[cam_idx])
54     AtA += eps * I3
55
56     # Solve the linear system for all vertices simultaneously
57     Xnxt = np.linalg.solve(AtA, Atb)
58
59     return Xnxt
60
61 def calculate_residuals(X_init, cam_idx, flows_fwd, intr, extr, shape):
62     """
63     Calculate residuals for Gauss-Newton optimization.
64     returns residual vector for the given vertex position
65     for one camera ray.
66     """
67
68     K = intr[cam_idx]
69     E = extr[cam_idx]
70     R = E[:, :3]
71     t = E[:, 3]
72     # Project X_init into camera i
73     Xc = R @ X_init + t
74     x, y, z = Xc
75     u = K[0, 0] * (x / z) + shape[cam_idx][1] * 0.5
76     v = K[1, 1] * (y / z) + shape[cam_idx][0] * 0.5
77
78     pix = np.array([v, u], dtype=np.float32).reshape(1, 2)
79     fwd, valid = query_at_image(flows_fwd[cam_idx], pix, return_valid=True)
80
81     if not valid[0]:
82         r = None
83     else:
```

```

84         vt, ut = (pix + fwd)[0].tolist()
85         r = np.array([v - vt, u - ut], dtype=np.float32)
86
87     return r
88
89
90 def compute_jacobian(X_init, cam_idx, intr, extr):
91     """
92     Compute the Jacobian matrix for a given camera.
93     """
94     K = intr[cam_idx]
95     E = extr[cam_idx]
96     R = E[:3, :3]
97     t = E[:3, 3]
98
99     # Project to camera space
100    Xc = R @ X_init + t
101    x, y, z = Xc
102
103    # Compute derivatives
104    du = np.array([K[0, 0] / z, 0, -K[0, 0] * x / (z * z)], dtype=np.float32)
105    dv = np.array([0, K[1, 1] / z, -K[1, 1] * y / (z * z)], dtype=np.float32)
106
107    # Transform to world space
108    Ju = du @ R
109    Jv = dv @ R
110    J = np.stack([Jv, Ju], axis=0)
111
112    return J
113
114 def compute_total_epe(X_cand, cams, flows_fwd, intr, extr, shape):
115     """
116     Compute end point errors, sum of 2D reprj errors for a candidate vertex
117     ↪ position.
118     """
119     epe = 0.0
120     for i in cams:
121         residual = calculate_residuals(X_cand, i, flows_fwd, intr, extr,
122                                       ↪ shape)
123         if residual is not None:
124             epe += np.linalg.norm(residual)
125     return epe
126
127 def refine_vertex_gn_eg_revised(X_init, cams, flows_fwd, intr, extr, shape):
128     """
129     Vertex refinement function that performs
130     One Gauss{Newton update with LM damping, step-clamping,
131     and a reprojection-error gate.
132
133     X_init:      np.ndarray,          # (3,) -- initial vertex position
134     cams:        list[int],           # inlier camera indices
135     flows_fwd:   list[np.ndarray],    # forward flows, already padded/resized
136     intr:        np.ndarray,          # (C,3,3) intrinsics

```

```
136     extr:         np.ndarray,          # (C,4,4) extrinsics
137     shape:        np.ndarray           # (C,2) image H×W
138     output:       np.ndarray           # (3,) --refined vertex position
139     """
140     # accumulate H and g into normal equations  $H \cdot dx = -g$ 
141     H = np.zeros((3, 3), dtype=np.float32)
142     g = np.zeros(3, dtype=np.float32)
143     for i in cams:
144         r = calculate_residuals(X_init, i, flows_fwd, intr, extr, shape)
145
146         if r is not None:
147             J = compute_jacobian(X_init, i, intr, extr)
148
149             H += J.T @ J
150             g += J.T @ r
151         else:
152             continue
153
154     # Levenberg{Marquardt damping
155     lambda_reg = 7e-1
156     H_reg = H + lambda_reg * np.eye(3, dtype=np.float32)
157
158     # solve normal eqn for refinement step dx
159     try:
160         dx = -np.linalg.solve(H_reg, g)
161     except np.linalg.LinAlgError:
162         return X_init # fallback to initial position if singular system
163
164     # clamp any exploding step
165     max_step = 0.005
166     norm_dx = np.linalg.norm(dx)
167     if norm_dx > max_step:
168         dx *= (max_step / norm_dx)
169
170     # fallback to initial position
171     if np.any(np.isnan(dx)) or norm_dx < 1e-8:
172         return X_init
173
174     # calculate epe before and after the update
175     epe_before = compute_total_epe(X_init, cams, flows_fwd, intr, extr, shape)
176     X_cand = X_init + dx
177     epe_after = compute_total_epe(X_cand, cams, flows_fwd, intr, extr, shape)
178
179     if epe_after > epe_before:
180         # reject harmful update
181         return X_init
182
183     # return the updated vertex
184     return X_cand
185
186
187 def load_gaussians(ply_path):
188     """
189     Returns:
```

```

190     X_g      (G,3) numpy { gaussian centres
191     n_g      (G,3) numpy { normals      (optional)
192     cov_g    (G,?) numpy { covariances (optional, not used in flow step)
193     """
194     pcd      = o3d.io.read_point_cloud(ply_path, remove_nan_points=True)
195     X_g      = np.asarray(pcd.points, dtype=np.float32)          # (G,3)
196     if len(pcd.normals) > 0:
197         n_g  = np.asarray(pcd.normals, dtype=np.float32)
198     else:
199         n_g  = None
200
201     if len(pcd.covariances) > 0:          # Open3D 0.18
202         cov = np.asarray(pcd.covariances, dtype=np.float32)
203     else:
204         cov = None
205
206     return X_g, n_g, cov
207
208
209 # GauSTAR helper functions -----
210
211
212 def pad_and_resize_flow(flow, pad, shape):
213     # pad = [(100, 20), (50, 50), (0, 0)]
214     pad = (np.append(pad, [0, 0])).reshape((-1, 2))
215     flow = np.pad(flow, pad_width=np.int32(pad), mode='constant',
216         ↪ constant_values=0)
217     scalar = shape[0] / flow.shape[0]
218     flow *= scalar
219     flow = cv2.resize(flow, shape[::-1], interpolation=cv2.INTER_NEAREST)
220     return flow
221
222 def query_at_image(image, pix, return_valid=False):
223     assert pix.shape[1] == 2
224     pix = np.int32(pix + 0.5)
225     shape = np.int32(image.shape[:2]) - 1
226     pix_clip = np.clip(pix, a_min=0, a_max=shape)
227
228
229     if return_valid:
230         valid_mask = (pix == pix_clip)
231         valid_mask = valid_mask[..., 0] & valid_mask[..., 1]
232         return image[pix_clip[:, 0], pix_clip[:, 1]], valid_mask
233     else:
234         return image[pix_clip[:, 0], pix_clip[:, 1]]
235
236
237 def points_to_local_points(points: np.ndarray, extr):
238     rot = extr[:3, :3]
239     trans = extr[:3, 3, None]
240     local_points = (np.matmul(rot, points.T) + trans).T
241     # pc = trimesh.Trimesh(vertices=local_points)
242     # pc.export("out/local_points.ply")

```

```
243     return local_points
244
245 def project(points: np.ndarray, intr, extr, shape, return_local_points=False):
246     """Projects a 3D point (x,y,z) to a pixel position (x,y)."""
247     batch_shape = points.shape[:-1]
248     points = points.reshape((-1, 3))
249
250     local_points = points_to_local_points(points, extr)
251     x = local_points[..., 0] / local_points[..., 2]
252     y = local_points[..., 1] / local_points[..., 2]
253
254     pixel_c = intr[0, 0] * x + shape[1] * 0.5 # col
255     pixel_r = intr[1, 1] * y + shape[0] * 0.5 # row
256     pixels = np.stack([pixel_r, pixel_c], axis=-1)
257
258
259     if return_local_points:
260         # depth = np.linalg.norm(local_points, axis=-1, keepdims=True)
261         return pixels.reshape((*batch_shape, 2)), local_points
262     else:
263         return pixels.reshape((*batch_shape, 2))
264
265 # new ray casting function
266 def pixel_to_local_rays(pixels: np.ndarray, intr, shape):
267     """
268     Given pixel coords (N×2), returns local ray directions (N×3) in camera
269     ↪ space.
270     """
271     x = (pixels[:, 1] - shape[1] * 0.5) / intr[0, 0]
272     y = (pixels[:, 0] - shape[0] * 0.5) / intr[1, 1]
273     dirs = np.stack([x, y, np.ones_like(x)], axis=-1)
274     norms = np.linalg.norm(dirs, axis=1, keepdims=True).clip(min=1e-6)
275     return dirs / norms
276
277 #-----
278
279 # Visualization and saving helper functions
280 ↪ -----
281
282 def visualize_flow_colormap(Xg, flow, save_dir, stats_csv_path, f_idx):
283     """
284     Visualize scene flow with colormap using pre-computed min/max from a CSV
285     ↪ log.
286
287     Xg                (G,3) array of point positions.
288     flow              (G,3) array of displacement vectors.
289     save_dir          directory in which to write the output image.
290     stats_csv_path    path to CSV that contains columns ['scene', 'min', 'max'].
291     f_idx              integer frame/scene index to look up in the CSV.
292     """
293     #Load the CSV (headers: 'scene', 'min', 'max')
294     df = pd.read_csv(stats_csv_path)
```

```

294
295 # Find the row where 'scene' == f_idx
296 row = df.loc[df['scene'] == f_idx]
297 if row.empty:
298     raise ValueError(f"No entry for scene ={f_idx} in {stats_csv_path!r}")
299
300 # Extract min+max for this frame
301 frame_min = float(row['min'].iloc[0])
302 frame_max = float(row['max'].iloc[0])
303 print(f"[scene {f_idx}] Using frame_min={frame_min:.8f},
    ↪ frame_max={frame_max:.8f}")
304
305
306 # Compute magnitudes & normalize using those CSV-provided bounds
307 epsilon = 1e-12
308 flow_mag = np.linalg.norm(flow, axis=1)
309 clipped = np.clip(flow_mag, frame_min, frame_max)
310 normed = (clipped - frame_min) / (frame_max - frame_min + epsilon)
311
312 # Map to RGB via matplotlib's \jet" colormap
313 cmap = plt.get_cmap("jet")
314 rgb_colors = cmap(normed)[:, :3] # drop the alpha channel
315
316 # Build an Open3D point cloud (color coded)
317 pcd = o3d.geometry.PointCloud()
318 pcd.points = o3d.utility.Vector3dVector(Xg)
319 pcd.colors = o3d.utility.Vector3dVector(rgb_colors)
320
321
322 # Render and save by taking a screenshot
323 save_image_path = os.path.join(save_dir, "scene_flow_visual.png")
324 vis = o3d.visualization.Visualizer()
325 vis.create_window(visible=True, width=1920, height=1080)
326 vis.add_geometry(pcd)
327 vis.poll_events()
328 vis.update_renderer()
329
330 # Let the window stay on screen for a moment to ensure it is captured
331 time.sleep(2.0)
332 # Capture the screen image and save it
333 vis.capture_screen_image(save_image_path)
334 vis.destroy_window()
335 print(f"[scene {f_idx}] Saved visualization to: {save_image_path}")
336
337
338 def export_scene_flow_as_ply(Xg, flow, save_path, stats_csv_path, f_idx):
339     """
340     Export a PLY with color-coded scene flow, using per-scene min/max from CSV
    ↪ log.
341     Xg                (G,3) array of point positions.
342     flow              (G,3) array of displacement vectors.
343     save_path         where to write the .ply file.
344     stats_csv_path    path to CSV that contains columns ['scene', 'min', 'max'].
345     f_idx             integer scene/frame index to look up in the CSV.

```

```
346     """
347     # Load the CSV
348     df = pd.read_csv(stats_csv_path)
349     row = df.loc[df['scene'] == f_idx]
350     if row.empty:
351         raise ValueError(f"No entry for scene={f_idx} in {stats_csv_path!r}")
352
353     frame_min = float(row['min'].iloc[0])
354     frame_max = float(row['max'].iloc[0])
355     print(f"[scene {f_idx}] Using CSV min={frame_min:.8f}, max={frame_max:.8f}")
356
357     # Compute magnitudes & normalize using scene bounds
358     flow_mag = np.linalg.norm(flow, axis=1)
359     clipped = np.clip(flow_mag, frame_min, frame_max)
360     norm_mag = (clipped - frame_min) / (frame_max - frame_min + 1e-8)
361
362     # Map normalized magnitudes to RGB
363     cmap = plt.get_cmap("jet")
364     colors = cmap(norm_mag)[: , :3] # drop alpha channel
365
366     # Build an Open3D point cloud with those colors
367     pcd = o3d.geometry.PointCloud()
368     pcd.points = o3d.utility.Vector3dVector(Xg)
369     pcd.colors = o3d.utility.Vector3dVector(colors)
370
371     # Write out the PLY
372     o3d.io.write_point_cloud(save_path, pcd)
373     print(f"[scene {f_idx}] Saved PLY with color-coded flow to: {save_path}")
374
375     # Config
376
377     class Config:
378         min_views          = 4          # require 4 cams to triangulate
379         bi_direct_pix_thresh = 4.0      # px threshold for forward/back check
380         eps                 = 1e-8      # small reg for the 3x3 system
381
382
383     # main warping function similar to GauSTAR's warping module
384     def warp_frame(f_idx, data_root, ply_root, out_root):
385         cfg = Config()
386         #load gaussians
387         ply_path = os.path.join(ply_root, f"{f_idx:04d}", f"{f_idx:04d}.ply")
388         Xg, ng, cov = load_gaussians(ply_path)
389
390         # Load mesh
391         mesh_path = os.path.join(ply_root, f"{f_idx:04d}", "coarse_mesh",
392                                   ↳ "warp_smooth.obj")
393         mesh = trimesh.load_mesh(mesh_path, process=False, maintain_order=True)
394         vert_num = mesh.vertices.shape[0]
395
396         # Load camera intrinsics/extrinsics
397         cam_data = np.load(os.path.join(data_root, "rgb_cameras.npz"))
398         intr, extr, shape = cam_data["intrinsics"], cam_data["extrinsics"],
399                                   ↳ cam_data["shape"]
```

```

398 C_orig = (-extr[:, :3, :3].transpose(0,2,1) @ extr[:, :3,
    ↪ 3, None]).squeeze(-1)
399
400 # Pre-allocate buffers
401 C = shape.shape[0]
402 flows_fwd, flows_bwd = [], []
403 rays_dir = np.zeros((C, vert_num, 3), dtype=np.float32)
404 rays_valid = np.zeros((C, vert_num), dtype=bool)
405
406
407 # Build rays from each camera
408 for cam_idx in range(C):
409     # load & pad/resize optical flow
410     pad = np.loadtxt(os.path.join(data_root, f"{f_idx:04d}", "flow_bi",
    ↪ "pad.txt"))
411     f_path = os.path.join(data_root, f"{f_idx:04d}", "flow_bi",
    ↪ f"{cam_idx:04d}_f.tiff")
412     b_path = os.path.join(data_root, f"{f_idx:04d}", "flow_bi",
    ↪ f"{cam_idx:04d}_b.tiff")
413     flow = tf.imread(f_path)[..., :-1] if os.path.exists(f_path) else
    ↪ np.load(f_path.replace('.tiff', '.npz'))["flow"][..., :-1]
414     flow_back = tf.imread(b_path)[..., :-1] if os.path.exists(b_path) else
    ↪ np.load(b_path.replace('.tiff', '.npz'))["flow"][..., :-1]
415     flow = pad_and_resize_flow(flow, pad, shape[cam_idx])
416     flow_back = pad_and_resize_flow(flow_back, pad, shape[cam_idx])
417     flows_fwd.append(flow); flows_bwd.append(flow_back)
418
419
420 # project & mask
421 pix_cur = project(mesh.vertices, intr[cam_idx], extr[cam_idx],
    ↪ shape[cam_idx])
422 fwd, ok1 = query_at_image(flow, pix_cur, return_valid=True)
423 pix_next = pix_cur + fwd
424 bwd, ok2 = query_at_image(flow_back, pix_next, return_valid=True)
425 mask = ok1 & ok2 & (np.linalg.norm(bwd + fwd, axis=1) <
    ↪ cfg.bi_direct_pix_thresh)
426
427 # form world-space rays
428 dirs_cam = pixel_to_local_rays(pix_next, intr[cam_idx], shape[cam_idx])
429 dirs_world = (extr[cam_idx, :3, :3].T @ dirs_cam.T).T
430 rays_dir[cam_idx] = F.normalize(torch.from_numpy(dirs_world),
    ↪ dim=1).numpy()
431 rays_valid[cam_idx] = mask
432
433 # Linear triangulation
434 Xnxt = linear_triangulation(rays_dir, rays_valid, C_orig, vert_num, cfg.eps)
435
436 # visibility count for debugging
437 vis_count = rays_valid.sum(axis=0)
438 print("vis_count: ", vis_count)
439
440
441 # Non-linear BA refinement
442 min_ba = max(cfg.min_views, 4)

```

```
443 min_ba = 3
444 for v in range(vert_num):
445     cams = np.where(rays_valid[:,v])[0].tolist()
446     if len(cams) < min_ba:
447         continue
448     Xnxt[v] = refine_vertex_gn_eg_revised(Xnxt[v], cams, flows_fwd, intr,
449     ↪ extr, shape)
449
450
451 """ # this is a possible post-processing step to filter outliers
452 # Neighbor-based outlier-clipped averaging
453 vert_flow = Xnxt - mesh.vertices
454 vert_filt = np.zeros_like(vert_flow)
455 min_obs = 3
456 sigma_thr = 3.0
457
458 for i, neis in enumerate(mesh.vertex_neighbors):
459     idxs = list(neis) + [i]
460     flows = vert_flow[idxs]
461     m, s = flows.mean(0), flows.std(0).clip(1e-6)
462     z = (flows - m) / s
463     keep = np.all(np.abs(z) < sigma_thr, axis=1)
464     keep[-1] = True
465     filt = flows[keep]
466     vert_filt[i] = filt.mean(0) if filt.shape[0] >= min_obs else
467     ↪ vert_flow[i]
468
469 """
470 # Save warped vertices + scene flow
471 out_dir = os.path.join(out_root, f"{f_idx:04d}")
472 os.makedirs(out_dir, exist_ok=True)
473
474 pcd_out = o3d.geometry.PointCloud()
475 pcd_out.points = o3d.utility.Vector3dVector(Xnxt)
476 if ng is not None: pcd_out.normals = o3d.utility.Vector3dVector(ng)
477 if cov is not None: pcd_out.covariances = o3d.utility.Matrix3dVector(cov)
478 o3d.io.write_point_cloud(os.path.join(out_dir,
479     ↪ f"vertices_warp_{f_idx:04d}.ply"), pcd_out)
480 scene_flow = Xnxt - mesh.vertices
481 np.save(os.path.join(out_dir, f"scene_flow_{f_idx:04d}.npy"), scene_flow)
482
483
484 ply_root = "root-to-GauSTAR-output-folder"
485 mesh_path = "root-to-GauSTAR-output-folder"
486 data_root = "root-to-GauSTAR-input-folder"
487 out_root = "root-to-prediction-output-folder"
488 os.makedirs(out_root, exist_ok=True)
489 stats_csv = "root-to-csv-file-containing-per-frame-scales"
490
491
492 for f_idx in range(21,171):
493     print("idx: ", f_idx)
494     warp_frame(f_idx, data_root, ply_root, out_root)
```

```
493 Xg = np.asarray(o3d.io.read_point_cloud(out_root +  
    ↪ f"{f_idx:04d}/vertices_warp_{f_idx:04d}.ply").points)  
494 flow = np.load(out_root + f"{f_idx:04d}/scene_flow_{f_idx:04d}.npz")  
495  
496 save_dir = os.path.join(out_root, f"{f_idx:04d}")  
497 os.makedirs(save_dir, exist_ok=True)  
498 visualize_flow_colormap(  
499     Xg=Xg,  
500     flow=flow,  
501     save_dir=save_dir,  
502     stats_csv_path=stats_csv,  
503     f_idx=f_idx  
504 )  
505 export_scene_flow_as_ply(Xg, flow,  
    ↪ save_path=out_root+f"{f_idx:04d}/scene_flow_colored.ply",  
    ↪ stats_csv_path=stats_csv, f_idx=f_idx)  
506
```

Bibliography

- [1] Xiaopeng Deng, Kai Yang, Yong Wang, Jinlong Li, and Liming Xie. A non-rigid point cloud registration method based on scene flow estimation. *IET Image Processing*, 19(1):e13308, 2025.
- [2] Quankai Gao, Qiangeng Xu, Zhe Cao, Ben Mildenhall, Wenchao Ma, Le Chen, Danhang Tang, and Ulrich Neumann. Gaussianflow: Splatting gaussian dynamics for 4d content creation, 2024.
- [3] Yi-Hua Huang, Yang-Tian Sun, Ziyi Yang, Xiaoyang Lyu, Yan-Pei Cao, and Xiaojuan Qi. Sc-gs: Sparse-controlled gaussian splatting for editable dynamic scenes, 2024.
- [4] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3d gaussian splatting for real-time radiance field rendering, 2023.
- [5] Yair Kittenplon, Yonina C. Eldar, and Dan Raviv. Flowstep3d: Model unrolling for self-supervised scene flow estimation, 2021.
- [6] Zhengqi Li, Simon Niklaus, Noah Snavely, and Oliver Wang. Neural scene flow fields for space-time view synthesis of dynamic scenes, 2021.
- [7] Jonathon Luiten, Georgios Kopanas, Bastian Leibe, and Deva Ramanan. Dynamic 3d gaussians: Tracking by persistent dynamic view synthesis, 2023.
- [8] Wei Mao, Richard Hartley, Mathieu Salzmann, and miaomiao Liu. Neural SDF flow for 3d reconstruction of dynamic scenes. In *The Twelfth International Conference on Learning Representations*, 2024.
- [9] Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. Nerf: Representing scenes as neural radiance fields for view synthesis, 2020.
- [10] René Schuster, Oliver Wasenmüller, Christian Unger, Georg Kuschik, and Didier Stricker. SceneFlow-fields++: Multi-frame matching, visibility prediction, and robust interpolation for scene flow estimation. *International Journal of Computer Vision*, 128(2):527–546, November 2019.
- [11] Zachary Teed and Jia Deng. Raft: Recurrent all-pairs field transforms for optical flow, 2020.
- [12] Kyle Vedder, Neehar Peri, Ishan Khatri, Siyi Li, Eric Eaton, Mehmet Kemal Kocamaz, Yue Wang, Zhiding Yu, Deva Ramanan, and Joachim Pehserl. Neural eulerian scene flow fields. In *The Thirteenth International Conference on Learning Representations*, 2025.
- [13] Peng Wang, Lingjie Liu, Yuan Liu, Christian Theobalt, Taku Komura, and Wenping Wang. Neus: Learning neural implicit surfaces by volume rendering for multi-view reconstruction, 2023.
- [14] Wenxuan Wu, Zhiyuan Wang, Zhuwen Li, Wei Liu, and Li Fuxin. Pointpwc-net: A coarse-to-fine network for supervised and self-supervised scene flow estimation on 3d point clouds, 2020.

- [15] Chengwei Zheng, Lixin Xue, Juan Zarate, and Jie Song. Gaustar: Gaussian surface tracking and reconstruction, 2025.
- [16] Ruijie Zhu, Yanzhe Liang, Hanzhi Chang, Jiacheng Deng, Jiahao Lu, Wenfei Yang, Tianzhu Zhang, and Yongdong Zhang. Motiongs: Exploring explicit motion guidance for deformable 3d gaussian splatting, 2024.